

PROJECT TITLE

Modular Expense Tracker

NAME

NAIVEDIYA JOSHI

REG.NO

26BAI11042

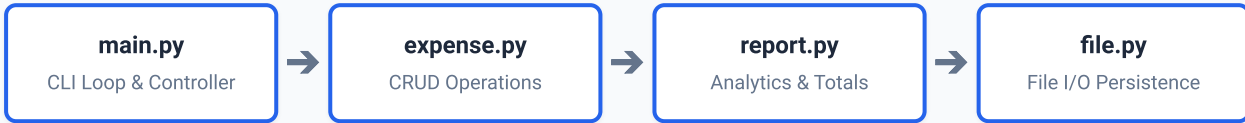
1. Project Overview & Objectives

The **Modular Expense Tracker Application** is designed as a minimalist, robust Command Line Interface (CLI) application built using standard Python libraries. The target objective is to enable individual users to record daily financial transactions, categorize expenditures, review categorical allocations, and store data persistently without relying on heavyweight external database management systems or third-party dependencies.

Key Design Focus: High modularity, zero external library dependencies, lightweight text-file persistence (`expenses.txt`), and clean separation of concerns across dedicated functional scripts.

2. System Architecture & Component Mapping

The system follows a clean modular architectural pattern, isolating user interactions, business logic, persistence operations, and reporting functionalities into specific source files:



Module File	Primary Function	Key Procedures
main.py	Menu Control	Main loop, choice evaluation, runtime execution sequence.
expense.py	Operations Management	add_expense(), show_expenses(), delete_expense()
file.py	Layer	save(), load() using pipe-delimited text serialization.
report.py	Financial Computation	total() sum aggregation, category_report() grouping.

3. Functional Features

Expense Recording Captures transaction amount, category name, and descriptive notes with strict runtime type casting and error handling.	Tabular Inspection Enumerates stored expenses linearly with index tracking for explicit item reference and deletion operations.
Safe Item Deletion Allows users to target specific entries via index. Includes bounds validation to prevent index out-of-range runtime errors.	Categorical Analysis Dynamically aggregates expenditure totals grouped by unique user-defined categories for instant budgeting insights.



5. Error Handling & Robustness

- **File Safety (FileNotFoundError):** The application gracefully initializes an empty list on startup if `expenses.txt` does not exist, preventing initial crash state.
- **Value Error Prevention:** Floating point conversion inputs (`float(input(...))`) are protected with `try-except` blocks to block non-numeric input crashes.
- **Index Bounds Verification:** Deletion routines verify index criteria (`1 <= n <= len(expenses)`) before modifying memory.

6. Installation & Execution Guide

1. Clone the project repository to your local directory:

```
git clone https://github.com/[Your-Username]/expense-tracker.git
```

2. Ensure Python 3.x is installed on your local environment.
3. Run the main application controller:

```
python main.py
```

